Introduction to Programming:

OPERATORS AND EXPRESSIONS

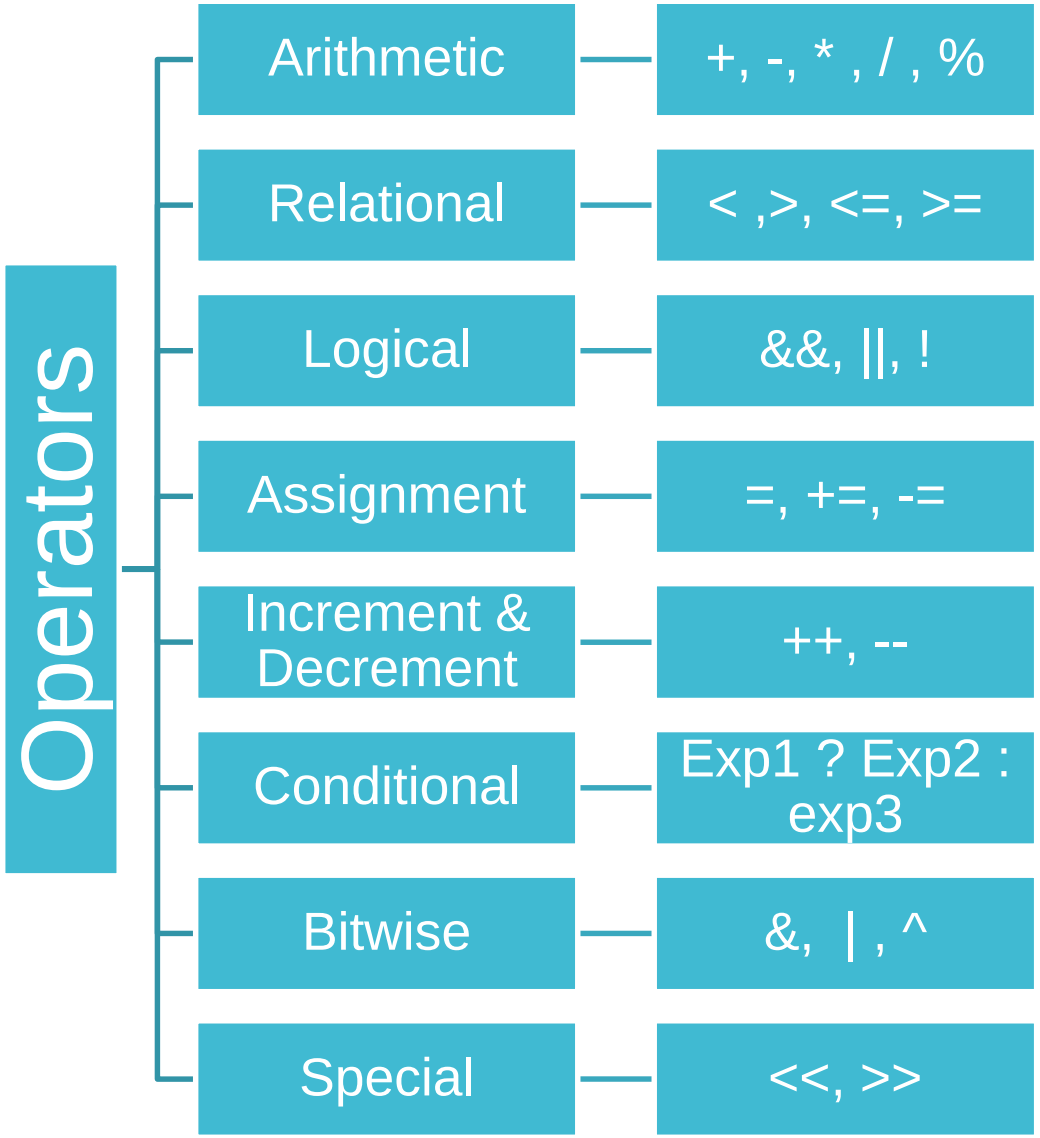
Prepared By:

Darsha Chauhan

Outline

- Operators:
  - Arithmetic
  - Relational
  - Logical
  - Assignment
  - Increment and Decrement
  - Conditional
  - Bitwise
  - Comma Operator
  - sizeof Operator
  - Operators used in Pointers and Structures
- Arithmetic Expressions
- How C programming Evaluates Arithmetic Expressions
- Precedence of Arithmetic Operators and Associativity Rule
- Type Conversion: Implicit and Explicit.

Operators


Operators
Arithmetic
+, -, * , / , %
Relational
< ,>, <=, >=
Logical
&&, ||, !
Assignment
=, +=, -=
Increment & Decrement
++, --
Conditional
Exp1 ? Exp2 : exp3
Bitwise
&, | , ^
Special
<<, >>

Operators: Arithmetic

- Arithmetic operators are used for mathematical calculations

Operator	Meaning	Example	Description
+	Addition	a + b	Addition of a & b
-	Subtraction	a – b	Subtraction of b from a
*	Multiplication	a * b	Multiplication of a & b
/	Division	a / b	Division of a by b
%	Modulo division-reminder	a % b	Modulo of a by b



Operators: Relational

- Relational operators are used to compare two numbers and taking decisions based on their relation.

Operator	Meaning	Example	Description
<	Is less than	a < b	a is less than b
<=	Is less than or equal to	a <= b	a is less than or equal to b
>	Is greater than	a > b	a is greater than b
>=	Is greater than or equal to	a >= b	a is greater than or equal to b
==	Is equal to	a = b	a is equal to b
!=	Is not equal to	a != b	a is not equal to b

Operators: Logical

- Logical operators are used to test more than one condition and make decisions.

Operator	Meaning
&&	logical AND (Both non zero then true, either is zero then false)
||	logical OR (Both zero then false, either is non zero then true)
!	logical NOT

A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Operators: Assignment

- Assignment operators (=) is used to assign the result of an expression to a variable.
- Assignment operator stores a value in memory.
- C also supports shorthand assignment operators which simplify operation with assignment.

Operator	Meaning
=	Assigns value of right side to left side
+=	a += 1 is same as a = a + 1
-=	a -= 1 is same as a = a - 1
*=	a *= 1 is same as a = a * 1
/=	a /= 1 is same as a = a / 1
%=	a %= 1 is same as a = a % 1



Operators: Increment & Decrement

- Increment (++) operator used to increase the value of the variable by one.
- Decrement (--) operator used to decrease the value of the variable by one.

Example	Explanation

x=100;
 	
x++;
	After the execution the value of x will be 101.

x=100; x--;	After the execution the value of x will be 99.

Operators: Increment & Decrement

- Increment (++) operator used to increase the value of the variable by one.
- Decrement (--) operator used to decrease the value of the variable by one.

Operator	Description	Eg	Explanation	output
Pre increment ++x	value of x is incremented before assigning it to the variable on the left	x=10; ++x;	First increment value of x by one then assign.	x will be 11 p will be 11

Post increment operator (x++)	value of x is incremented after assigning it to the variable on the left	x=10;
		
		p=x++;
			First assign value of x then increment value.	x will be 11
				
				p will be 10



Operators: Ternary / Conditional

- A ternary operator is known as conditional operator.
- Syntax: exp1 ? exp2 : exp3

Working of the ? : Operator

- exp1 is evaluated first
- if exp1 is true(nonzero) then
  - - exp2 is evaluated and its value becomes the value of the expression
- If exp1 is false(zero) then
  - - exp3 is evaluated and its value becomes the value of the expression
- For example:

  m=2, n=3;
  r=(m>n) ? m : n;
  Value of r will be 3



Operators: Bitwise

Operator	Meaning	Example	Output

&	Bitwise AND	int a=8, b=6, c;
		c = a & b;
		printf("Output = %d", c);
			0

|	Bitwise OR	int a=8, b=6, c;
		c = a | b;
		printf("Output = %d", c);
			14

^	Bitwise exclusive OR	int a=8, b=6, c;
		c = a ^ b;
		printf("Output = %d", c);
			14

<<	Shift left	int a=8, b;
		b = a << 1;
		printf("Output = %d", b);
			16

>>	Shift right	int a=8, b;
		b = a >> 1;
		printf("Output = %d", b);
			4

Operators: Special

Operator	Meaning
&	Address operator, it is used to determine address of the variable.
*	Pointer operator, it is used to declare pointer variable and to get value from it.
,	Comma operator. It is used to link the related expressions together.
sizeof	It returns the number of bytes the operand occupies.
.	member selection operator, used in structure.
->	member selection operator, used in pointer to structure.



Expression

- An expression is a combination of operators, constants and variables.
- An expression may consist of one or more operands, and zero or more operators to produce a value.
- For Example: sum = a + b + c, area = 3.14 * r *r

Evaluation of an expression

- An expression is evaluated based on the operator precedence and associativity.
- When there are multiple operators in an expression, they are evaluated according to their precedence and associativity.



Expression: Precedence

- Precedence of an operator is its priority in an expression for evaluation.
- The operator with higher precedence is evaluated first and the operator with the least precedence is evaluated last.
- Operator precedence
  - 5 + 3 * 2 = 5 + (3 * 2) = 11, and
  - Not as (5 + 3) * 2= 16.
- We say that the multiplication operator (*) has higher "precedence" or "priority" than the addition operator (+), so the multiplication must be performed first.



Expression: Associativity

- Associativity is the left-to-right or right-to-left order for grouping operands to operators that have the same precedence.
- Operator associativity
  - 8 - 3 – 2 = (8 - 3) – 2 = 3
  - Not as 8 - (3 - 2)= 7.
- We say that the subtraction operator (-) is "left associative", so the left subtraction must be performed first.
- When we can't decide by operator precedence alone in which order to calculate an expression, we must use associativity.

Expression: Associativity

Operator	Description	Associativity
() [ ] . -> ++ --	Parentheses: grouping or function call Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Expression: Associativity

Operator	Description	Associativity
() [ ] . -> ++ --	Parentheses: grouping or function call Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Type Conversion

- Type conversion is converting one type of data to another type.
- It is also known as Type Casting.
- There are two types of type conversion:
- Implicit Type Conversion:
  - This type of conversion is usually performed by the compiler when necessary without any commands by the user.
  - It is also called Automatic Type Conversion.
  - If operand are of different types, the lower type is automatically converted to the higher type before operation proceeds
  - For Example:

```
int a = 20;
double b = 20.5;
printf("%lf", a + b);
```



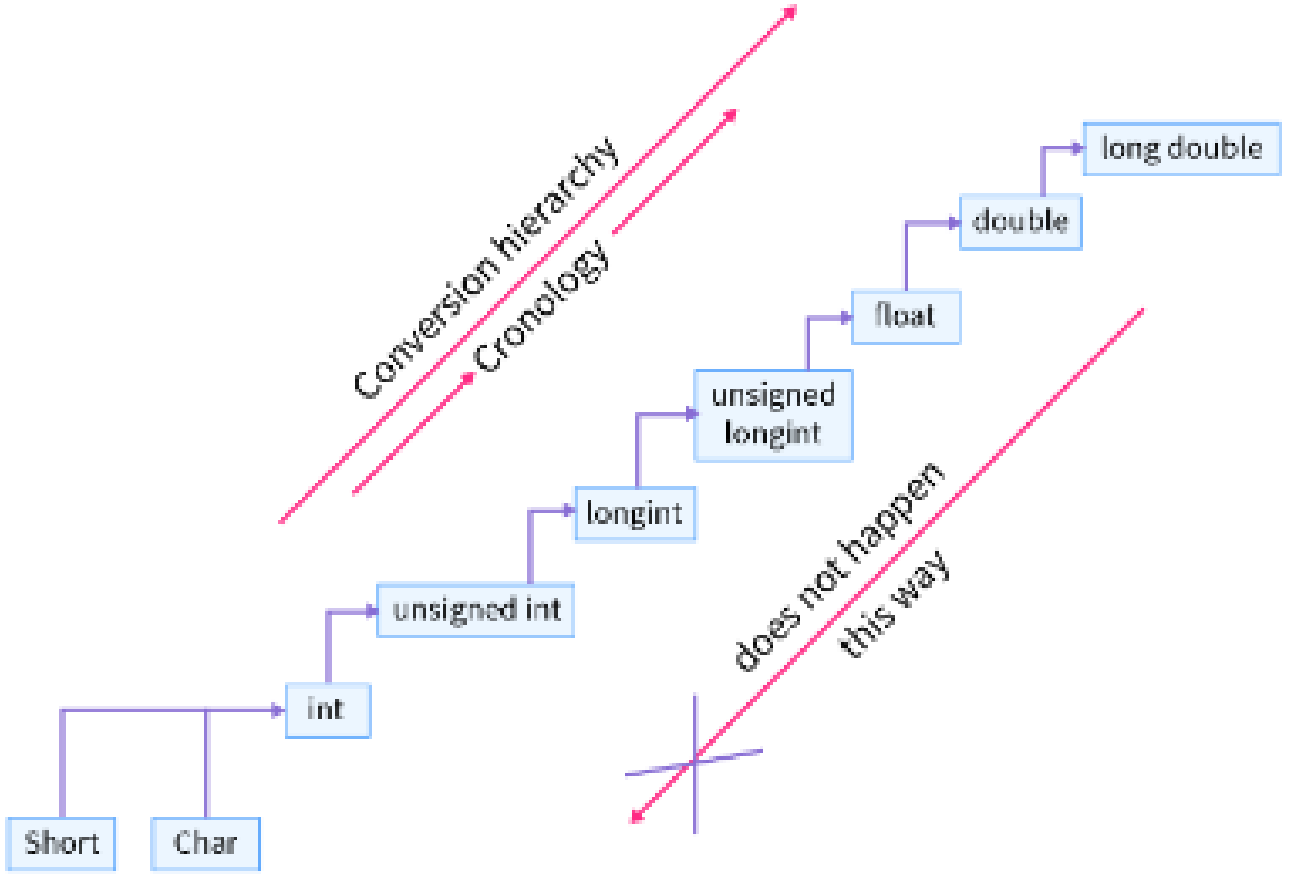
Type Conversion

- Explicit Type Conversion:
  - These conversions are done explicitly by users using the pre-defined functions.
  - For Example:

```
double a = 4.5, b = 4.6, c = 4.9;
int result = (int)a + (int)b + (int)c;
printf("result = %d", result);
```



Type Conversion



#Thank You

From @:Darsha

☺☺☺